Artifact Appendix: Vista: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing

Submission ID #217

1 Artifact Overview

This artifact supports the CAIS 2026 paper "Vista: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing." The artifact contains the training code for Vista, the staged OpenQASM verifier used during agentic reinforcement learning, generation scripts for Vista and baseline checkpoints, and offline evaluation scripts for reproducing the reported semantic metrics.

The main artifact components are:

- `examples/train/quantum/`: GRPO training scripts for the 4B quantum circuit model.
- `verl_tool/servers/tools/quantum_cpu.py`: the quantum verifier service used during tool-augmented rollouts.
- `verl_tool/servers/tools/utils/quantum_reward_cal.py`: staged reward computation for syntax feasibility, behavior similarity, expectation-value objective quality, and utility-oriented optimization.
- `quantum-code-generation/code/generation/`: scripts for generating OpenQASM circuits from Vista and baseline models.
- `quantum-code-generation/code/evaluation/`: scripts for parsing generated QASM, simulating circuits, computing relative entropy, checking expectation values, and writing summary statistics.
- `quantum-code-generation/code/data_generation/src/algorithms/`: graph-generation source files and included pickle inputs for the supported graph optimization primitives.
- `artifact/scripts/run_quantum_experiment.sh`: reviewer wrapper for generation plus evaluation, or evaluation of an existing raw generation file.
- `artifact/scripts/draw_vista_figures.py` and `vista_draw/`: headless wrappers and plot-ready inputs for regenerating paper figures.
- `artifact/tables/` and `artifact/scripts/show_paper_tables.py`: extracted CSV values for Tables 1–3 and a dependency-free Markdown display script.
- Released checkpoints and data on Hugging Face: `Benyucong/rl_quantum_4b`, `Benyucong/sft_quantum_circuit_gen_4B`, and `Benyucong/graph-data-quantum-rl`.

2 Badge Target

We request the Functional and Results Reproduced badges for a GitHub-only submission at https://github.com/benyucong/rl-quantum. CAIS's Available checklist requires a public archive with irrevocable versioning and long-term storage, such as Zenodo but not GitHub, so we do not rely on the Available badge unless a DOI-backed archive is added later. The repository includes public checkpoint and dataset links, and the final submitted version should include raw generated-circuit JSON files for all reported model runs plus precomputed logs/results.

3 Hardware and Software Requirements

The minimal artifact inspection path requires Python 3.10 or 3.11 and does not require a GPU. Recomputing evaluation metrics from raw generated circuits requires the Python packages listed in `quantum-code-generation/code/evaluation/requirements.txt`, including Qiskit 1.2.4, qiskit-aer 0.16.1, and qiskit-qasm3-import 0.5.1.

Regenerating model outputs requires a GPU-capable machine with enough memory for the selected 3B/4B model checkpoint. Full Vista retraining is substantially more expensive: the paper-scale configuration fine-tunes a 4B model with GRPO and FSDP on 128 AMD GPUs for approximately 48 hours. The artifact also includes smaller single-node Slurm wrappers intended for sanity checks or partial reproduction.

4 Major Claims and Reproduction Plan

5 Evaluation Workflow

The quickest artifact check is:

```
cd /path/to/rl-quantum
python3 artifact/scripts/summarize_eval_outputs.py \
  quantum-code-generation/code/evaluation/out
python3 artifact/scripts/show_paper_tables.py all
```

These commands parse the included precomputed summary files and print the extracted paper Tables 1–3 as Markdown.

For a fresh GPU-backed run followed by plots derived from the new evaluation output, evaluators can use:

```
bash artifact/scripts/run_quantum_experiment.sh \
  --n-samples 50 \
  --out-dir artifact_runs/reviewer_smoke
EVAL_DIR=artifact_runs/reviewer_smoke/evaluation
FIG_DIR=artifact_runs/reviewer_smoke/figures
python3 artifact/scripts/draw_vista_figures.py \
  --evaluation-dir "$EVAL_DIR" \
  --only box,relative_entropy \
  --output-dir "$FIG_DIR" \
  --strict
```

The first command writes raw generations and `summary_stats_VISTA.json`/`summary_VISTA_raw_data.csv`. The second command converts that new evaluation CSV into the objective-gap and relative-entropy plot inputs and runs the existing figure scripts. The remaining paper figures can be redrawn from packaged plot-ready inputs with `draw_vista_figures.py--input-dirvista_draw`.

To recompute metrics from raw generated outputs, evaluators should run:

```
cd quantum-code-generation/code/evaluation
python3 -m venv .venv-eval
source .venv-eval/bin/activate
pip install -r requirements.txt
GEN_JSON=../generation/out/<RAW_GENERATION_JSON>
```

Claim	Paper result	Artifact path	Reproduction method
End-to-end effectiveness	Vista reaches Pass@1 SCR 99.31%, SREV 22.41%, RE 11.61, HQCR 17.24%; and Pass@10 SCR 100%, SREV 33.10%, RE 8.48, HQCR 27.24%.	`quantum-code-generation/code/evaluation/src/evaluate_samples.py`; raw generation JSONs; `evaluation/out`.	Run the evaluator on each raw generation file and compare `summary_stats_*.json` against Table 1.
Reward ablation	Full Vista outperforms w/o EV, w/o RE, w/o Opt, and validity-only ablations; removing RE causes the largest drop.	`verl_tool/servers/tools/utils/quantum_reward_cal.py`; ablation raw outputs.	Evaluate raw ablation generations with the same evaluator and compare against Table 2.
Verifier efficiency	Gated evaluation reduces verifier time from 8.20s to 4.63s per iteration, a 1.77x speedup.	Training and verifier timing logs; reward/verifier scripts.	Inspect archived timing logs or rerun gated and non-gated variants under the same verifier budget.
Scaling and robustness	Vista remains stronger across semantic thresholds, graph primitives, and 8–16 qubit problem sizes.	Raw generations with task metadata; evaluation outputs.	Group evaluation results by primitive, threshold, qubit count, and gate count.
Hardware deployment	Vista preserves objective quality while reducing scheduled circuit duration on real or emulated QPU backends.	Archived hardware logs and transpiled circuits.	Inspect hardware logs or rerun the hardware postprocessing scripts if QPU access is available.

Table 1: Claim-to-artifact mapping for the Vista artifact.

```
python3 src/evaluate_samples.py \
  "$GEN_JSON" \
  out \
  <MODEL>
```

The command writes `summary_stats_<MODEL>.json` and `summary_<MODEL>_raw_data.csv`. These outputs should be compared with the corresponding paper table or figure.

To regenerate model outputs, evaluators with GPU access can run:

```
cd quantum-code-generation/code/generation
python3 -m venv .venv-gen
source .venv-gen/bin/activate
pip install -r requirements.txt
python3 generate_samples.py \
  --uid cais26 \
  --model_path Benyucong/rl_quantum_4b \
  --dataset Benyucong/graph-data-quantum-rl \
  --n_samples 200
```

6 Notes for the AEC

Full retraining is not expected for kick-the-tires because it requires a large GPU cluster. The intended reproducibility path is to use the released checkpoint and raw generation outputs, then rerun the deterministic evaluation scripts. Real-QPU experiments depend on external hardware availability and should be evaluated from archived hardware logs unless the AEC explicitly requests a live run.

The current artifact includes graph-generation pickle inputs, precomputed summary JSON/CSV files, the public RL checkpoint `Benyucong/rl_quantum_4b`, the public SFT checkpoint `Benyucong/sft_quantum_circuit_gen_4B`, and the public dataset `Benyucong/graph-data-quantum-rl`. For a low-friction Results Reproduced evaluation, the final archive should also include the raw generated-circuit JSON files used as inputs to `evaluate_samples.py`.

The submitted artifact URL is https://github.com/benyucong/rl-quantum. If a DOI-backed archive is added later, list that archive URL in HotCRP as well.